

Gobernanza, Roadmap y Despliegue - Sprint 2

Futsi Mini ERP | Actualizado 2026-05-28

Decision de gobierno: cerrar Sprint 2 completo en local antes de reintentar despliegue productivo, porque se agregaron modelos, endpoints, facturacion simulada, DeepFace local y cambios de infraestructura.

1. Estado del producto

Elemento	Estado	Evidencia
Excel contable real	Hecho	GET /api/reports/accounting.xlsx genera workbook valido con Resumen, Pagos, Cargos, Gastos, Descuentos, Asistencia con adeudo y Facturas.
Importacion historica Excel	Hecho base	Admin/contador cargan Excel, capturan password si aplica, revisan preview editable, firman y generan pagos/gastos historicos con auditoria.
Facturacion PAC simulada	Hecho	Modelo Invoice, endpoint /api/invoices/simulate/, descarga PDF y XML con UUID simulado.
Facturas en perfiles	Hecho	Admin/contador generan; guardian/coach/cajero consultan segun permisos.
Pase de lista facial	Hecho local	DeepFace instalado en .env; endpoint diagnostico y reconocimiento con recuadro azul/verde/rojo.
Alumno demo con foto real	Hecho local	Marco Antonio Demo en Roma / Equipo Sub-12 A con foto students/photos/retrato_marco.jpeg.
PWA Android gratis	Hecho base	Manifest, icono SVG, service worker y registro en produccion.
APK Android Capacitor	Hecho demo	APK debug generado en front/android/app/build/outputs/apk/debug/app-debug.apk y consume API local via 10.0.2.2.
Responsive mobile	Hecho base	CSS global para header, tablas, formularios, botones tactiles y overflow.
Tema oscuro web/Android	Hecho	Toggle global claro/oscuro con persistencia localStorage, estilos de cards, tablas, inputs, mapa y menu movil.
Despliegue frontend	Hecho	GitHub Pages publica React en /futsi/ con VITE_API_URL a Render.
Despliegue backend	Preparado / pausado	Render + Dockerfile + render.yaml + Supabase Postgres; produccion final se reintentara al cerrar Sprint 2.
Supabase Postgres	Preparado	settings.py soporta POSTGRES_* con sslmode=require para evitar errores por URL malformada.

2. Solucion de despliegue documentada

Problema	Causa	Solucion aplicada
GitHub Pages mostraba documentacion o pagina incompleta	Pages solo sirve estaticos y estaba apuntando a salida/branch incorrecto.	Workflow de frontend construye Vite y publica front/dist en gh-pages; base path /futsi/.
Login devolvía Unexpected token '<'	Frontend llamaba una ruta HTML en lugar del API JSON.	VITE_API_URL apunta a https://futsi.onrender.com/api y el cliente valida respuesta.

Problema	Causa	Solucion aplicada
Backend no podia vivir en Pages	GitHub Pages no ejecuta procesos Python ni gunicorn.	Django se hospeda en Render como Web Service; Pages queda solo para React.
Render fallaba con ValueError port 'H'	SUPABASE_DATABASE_URL contenia password sin URL encoding, rompiendo parseo de puerto.	settings.py prioriza POSTGRES_* separados: host, puerto, usuario, password, db y sslmode.
Produccion quedo inestable durante cambios de Sprint 2	Se estaban moviendo modelos, dependencias y endpoints nuevos.	Decision de gobierno: seguir local hasta cerrar Sprint 2 y desplegar en bloque probado.

3. Roadmap de sprints

Sprint	Objetivo	Entregables	Criterio de salida
Sprint 1	Fuente confiable de operacion.	Login, sedes, alumnos, asistencia, pagos, gastos, dashboard, auditoria basica.	Demo de cruce asistencia vs cobranza vs gastos.
Sprint 2	Contabilidad, despliegue y movilidad demo.	Excel contable, PAC simulado, PWA, DeepFace local, Render/Supabase preparado.	Demo e2e local estable y documentos actualizados.
Sprint 3	Produccion controlada, historico y seguridad.	Migraciones Supabase, deploy Render, Pages final, importacion historica Excel ampliada, QA automatizado, Sonar, OWASP, storage, backups y monitoreo.	URL productiva estable, historico conciliado, quality gate aprobado y pruebas de seguridad sin hallazgos criticos.
Sprint futuro	Automatizacion avanzada.	SPEI real, terminal real, WhatsApp, OCR, camaras en sede, inventario y modulo deportivo avanzado.	Integraciones con proveedores contratados.

4. Gantt operativo propuesto

Semana	Trabajo	Responsable	Dependencias
2026-05-25 a 2026-05-31	Sprint 1 + cierre funcional demo.	Desarrollo	Datos demo y reglas base.
2026-06-01 a 2026-06-07	Sprint 2 local: contabilidad, facturacion simulada, PWA y DeepFace.	Desarrollo + usuario	Fotos demo, pruebas de Excel/PDF/XML.
2026-06-08 a 2026-06-12	Hardening y deploy: Supabase, Render, Pages, variables, smoke tests.	Desarrollo + admin cloud	Password DB, secretos GitHub/Render.
2026-06-10 a 2026-06-14	Importacion historica Excel: perfilado, mapeo, staging, validacion y conciliacion.	Desarrollo + contador	Archivo historico, diccionario de claves y criterios de limpieza.
2026-06-12 a 2026-06-16	Seguridad y QA: Sonar, pytest/Django, Playwright Python, pruebas agresivas de formularios y OWASP ZAP baseline.	Desarrollo + QA	Entorno staging, usuarios por rol y datos anonimizados.
2026-06-15 a 2026-06-21	Piloto con usuarios de una sede, APK Android y datos historicos conciliados.	Operacion + contador	Usuarios reales, historico importado, reglas aprobadas y APK validado.

5. Riesgos y controles

- DeepFace no debe tratarse como control definitivo en produccion sin consentimiento, pruebas de sesgo, camaras controladas y politicas de datos biometricos.
- Supabase debe recibir migraciones solo despues de validar localmente; no ejecutar seed_demo --reset en datos reales.
- El password de Postgres no debe guardarse en GitHub; en Render usar POSTGRES_PASSWORD como secreto.
- Facturacion actual es simulada; no sustituye PAC real ni CFDI fiscal.
- Efectivo requiere cierre de caja y aceptacion del cliente para reducir disputas, pero no elimina totalmente riesgo operativo.
- Antes de produccion se deben ejecutar pruebas de seguridad contra inyeccion SQL, acceso indebido por rol, cargas maliciosas y secretos expuestos.
- La importacion historica puede contaminar reportes si no pasa por staging, conciliacion y firma del contador/admin.

6. Sprint 3: checklist de seguridad, QA y cierre

Pendiente Sprint 3	Motivo	Evidencia esperada
Hardening de seguridad	La demo ya toca pagos, menores, fotos, datos medicos y facturas.	Matriz de amenazas, pruebas de permisos y reporte OWASP/Sonar.
QA automatizado	El proyecto ya tiene muchos roles y flujos; probar manualmente no escala.	CI ejecuta Django tests, Playwright y build web/APK.
Sonar	Controlar calidad, duplicacion, bugs y deuda tecnica.	Quality Gate aprobado en GitHub Actions.
Importacion historica ampliada	El primer parser cubre hojas de ingresos/gastos; falta historico completo y staging.	Excel antiguo cargado en staging, conciliado y firmado.
Despliegue productivo final	Sprint 2 quedo local por cambios fuertes.	Render + Supabase + Pages con smoke test y backup.
Android estabilizado	APK debug existe; falta ciclo formal de QA movil.	APK probado en emulador y celular fisico.

Area	Accion Sprint 3	Criterio de salida
SQL injection	Revisar queries crudas, filtros, reportes y parsers; mantener ORM parametrizado y pruebas con payloads maliciosos.	Suite automatizada confirma que filtros/login/importacion no ejecutan SQL no esperado.
Autenticacion	Validar sesiones/token, expiracion, logout, cambio de password y bloqueo de fuerza bruta.	Casos negativos cubiertos y logs de intentos fallidos.
Autorizacion por rol	Pruebas de acceso horizontal/vertical para admin, contador, cajero, coach, tutor y coordinador.	Cada endpoint sensible tiene test de permisos.
Carga de archivos	Validar extension, MIME, tamano, password, hojas permitidas y limpieza de Excel historico.	Archivos corruptos/enormes/protegidos fallan de forma controlada.
Datos personales	Revisar fotos, datos medicos, responsivas y facturas; minimizar exposicion por rol.	Cajero/coach/tutor solo ven datos necesarios.
CORS/CSRF	Endurecer origenes permitidos, headers, HTTPS y cookies si se cambia estrategia de auth.	Produccion no acepta origenes comodin.
Secretos	Validar Render/GitHub/Supabase sin secretos en repo.	git-secrets/trufflehog o equivalente sin hallazgos criticos.

Area	Accion Sprint 3	Criterio de salida
Dependencias	Ejecutar npm audit/pip-audit y fijar actualizaciones criticas.	Sin vulnerabilidades altas conocidas o con excepcion documentada.

Herramienta	Uso propuesto	Resultado esperado
SonarQube / SonarCloud	Analisis estatico de frontend/backend en GitHub Actions: bugs, smells, cobertura, duplicacion y hotspots.	Quality Gate visible por PR/main.
pytest + Django	Tests unitarios/API para modelos, permisos, pagos, facturas, importacion Excel, reportes y auditoria.	Suite backend reproducible en CI.
Playwright Python	Pruebas e2e usando elementos reales de la web: login por rol, navegacion movil, pagos, asistencia, Excel historico y facturas.	Flujos criticos pasan en Chrome headless.
Pruebas agresivas UI	Scripts que llenan formularios con valores largos, caracteres raros, fechas invalidas, montos negativos, clicks rapidos y reloads.	La app no se rompe visualmente ni guarda datos invalidos.
OWASP ZAP baseline	Escaneo automatizado contra entorno staging local/Render.	Reporte sin riesgos altos antes de piloto.
Lighthouse	Revisar PWA, accesibilidad y performance movil.	PWA instalable y sin problemas severos de accesibilidad.
Android smoke	Instalar APK debug en emulador y validar login, menu, tema oscuro y camara.	APK usable antes de compartir demo.

6. RACI operativo

Actividad	Responsable	Aprueba	Consultado	Informado
Reglas de mora/descuento	Administrador	Direccion	Contador / coordinadores	Cajeros / familias
Captura de pago	Cajero	Sistema / cliente en efectivo	Contador	Familia / representante
Autorizacion para jugar con adeudo	Coordinador	Coordinador / direccion segun tope	Cajero	Contador
Gasto operativo	Coordinador	Contador / direccion	Proveedor	Direccion
Cierre de caja	Cajero	Contador	Coordinador	Direccion
Deploy productivo	Desarrollo	Direccion tecnica	Usuario negocio	Operacion

7. Definition of Done para Sprint 2

- Migraciones locales aplicadas sin errores.
- Seed demo corre y deja usuarios/alumnos/cargos/gastos suficientes para demo.
- Excel contable abre en Excel/LibreOffice y contiene todas las hojas esperadas.
- Factura simulada se guarda en DB y permite descarga PDF/XML.
- Coach puede pasar lista manual y facial en local con feedback visual.
- PWA instala en Android desde Chrome.
- Documentacion SRS, gobernanza, codigo, negocio, despliegue y DeepFace actualizada.

- Produccion no se reintenta hasta que el usuario apruebe cierre local de Sprint 2.

8. Runbook de despliegue Sprint 3

- 1 Congelar rama main con Sprint 2 probado localmente.
- 2 Respalidar base local/demo y confirmar que no se ejecutara seed destructivo en produccion.
- 3 Crear/confirmar password de database en Supabase.
- 4 Configurar POSTGRES_* y secretos Django en Render.
- 5 Perfilar Excel historico y crear diccionario de columnas, claves, sedes y conceptos.
- 6 Importar historico primero a tablas staging para validar totales antes de afectar tablas operativas.
- 7 Conciliar ingresos, egresos, gastos y utilidad contra ESTADO DE RDOS., INGRESOS SEDES y GASTOS SEDES.
- 8 Ejecutar deploy Render y revisar logs de migracion.
- 9 Probar /health/ y /api/auth/login/ desde Render.
- 10 Actualizar variables GitHub Actions: VITE_API_URL y VITE_BASE_PATH.
- 11 Publicar Pages y probar login real desde la URL publica.
- 12 Descargar Excel, generar factura simulada, crear pago demo y validar reportes historicos.
- 13 Registrar evidencia de smoke test, conciliacion historica y aprobar piloto.

9. Monitoreo minimo

- Render logs para errores 500, migraciones y CORS.
- Supabase dashboard para conexiones, tablas, crecimiento y consultas lentas.
- GitHub Actions para build fallido de frontend/backend.
- Bitacora funcional: facturas emitidas, intentos faciales, pagos en proceso y gastos pendientes.

10. Gobierno de migracion historica

- Ningun Excel historico debe importarse directo a tablas finales sin pasar por staging.
- Cada carga debe registrar archivo, hash, usuario, fecha, hojas procesadas, filas leidas, filas rechazadas y totales por sede/mes.
- El contador debe aprobar la conciliacion entre origen Excel y datos normalizados antes de activar reportes historicos.
- Las diferencias deben clasificarse como formula rota, duplicado, falta de sede, concepto desconocido, fecha invalida, responsable no identificado o monto inconsistente.
- El sistema debe permitir reimportar de forma idempotente sin duplicar pagos/gastos.